Pruebas pre-energización — robot rearmado

Creado 2026-09-13 · protoboard rearmado 2026-09-12 · 2× BTS7960 · Buck nuevo 5.01V · a ejecutar **antes** de V3 y antes de medir `K_U`

⛔ **Por qué existe este documento.** Este robot ya falló **tres veces** con el mismo síntoma ("el firmware reporta todo bien y ningún motor se mueve": pin lógico sin alimentar, sin GND común, switch en OFF), **perdió un Buck, dos capacitores y un fusible** por polaridad invertida, y el 2026-09-12 se armó un **lazo de masa** que solo se detectó cruzando documentación. El cableado es nuevo de punta a punta y hay **dos cambios que nunca se probaron**: el reordenamiento M1→derecha / M2→izquierda y el paso de `RPWM` M1 de `P14` a `P19`.

Orden y criterio. Las etapas van de lo más barato y seguro a lo más riesgoso, y cada prueba está pensada para que **solo pueda fallar por un motivo**. No saltear etapas: la gracia es que cuando algo falle, ya sepas qué es.

Multímetro en modo Ω, no en continuidad/beep. El beep dispara con cualquier cosa por debajo de ~20-50Ω y no distingue nada a esa escala — es la lección 4 del proyecto.

Etapa 0 · Sin batería · multímetro en Ω

BATERÍA DESCONECTADA (XT60 fuera) · SWITCH EN OFF · nada energizado

0.1 — POLARIDAD DEL TRAMO HACIA EL BUCK

Esta es la prueba que se saltó el 2026-09-10 y costó un Buck, dos capacitores y el fusible. Hacerla siempre, sin importar cuántas veces haya salido bien — lección 11.

Con el XT60 **desenchufado**, medir continuidad desde el pin `+` del conector de batería hacia cada uno de los dos cables que llegan a la bornera `IN` del Buck.

MEDICIÓN	ESPERADO	SI FALLA
☐ XT60 `+` ↔ cable en `IN+`	continuidad	**Están invertidos.** Corregir antes de seguir. No confiar en el color del cable — el proyecto no usa código de colores a propósito.
☐ XT60 `−` ↔ cable en `IN−`	continuidad	

0.2 — LAS 6 PULL-DOWNS DE SEGURIDAD

De cada línea de señal al riel `−`. Valida a la vez que la resistencia está puesta **y** que está en la fila correcta.

LÍNEA	FILA / R	ESPERADO EN CIRCUITO	LECTURA
☐ `EN` M1 (`R_EN` + `L_EN`)	f.10 / J10	≈5.7–6kΩ	
☐ `LPWM` M1	f.8 / J8	≈7.45kΩ	
☐ `RPWM` M1	f.6 / J6	≈7.45kΩ	
☐ `EN` M2	f.9 / A9	≈5.7–6kΩ	
☐ `LPWM` M2	f.7 / A7	≈7.45kΩ	
☐ `RPWM` M2	f.5 / A5	≈7.45kΩ	

⚠️ **No se espera ≈10kΩ** (corregido 2026-09-14, registro §7.27). Medido en circuito con los drivers conectados, cada entrada del IBT-2 suma **≈26kΩ hacia GND** en paralelo: 9.9kΩ ∥ 26kΩ ≈ 7.45kΩ. `EN` tiene dos entradas atadas (≈13kΩ), por eso da menos. La resistencia suelta mide 9.9kΩ.

Cómo medir: la lectura arranca más baja y sube hasta estabilizarse — anotar el valor estable y descargar el nodo entre mediciones.

Cómo leer un resultado raro: abierto = falta la resistencia, está en la fila equivocada o no llega a GND. **Estable y claramente menor** = dos pull-downs en paralelo (≈4.3kΩ en `RPWM` / `LPWM`, ≈3.6kΩ en `EN`).

0.3 — CONTINUIDAD PUNTA A PUNTA: ESP32 → CADA DISPOSITIVO

Es el **punto 4 del checklist de V1, que quedó pendiente**. Cubre los tres grupos: **drivers, encoders y MPU6050**.

Medir siempre en el pin del ESP32 y en el pin del dispositivo, nunca en la fila del protoboard. Todo el punto de la prueba es detectar un contacto flojo de protoboard — es lo que arruinó el enlace UART del ESP8266 (mover el protoboard sin tocar un cable cambiaba el resultado) y el jumper de VIN de §3.2.

Poner el multímetro en cero antes de empezar. Acá se miden **cables**, y un cable sano da **menos de 1–2Ω**. Muchos multímetros tienen ~1Ω de offset propio, así que hay que cortocircuitar las puntas, anotar esa lectura y restarla. Sin esto, "1.8Ω" no se distingue de "0.8Ω" y no sabés si el contacto es bueno.

⛔ **Modo Ω, no continuidad/beep.** El beep pita con cualquier cosa por debajo de ~20–50Ω: no distingue un cable sano de un contacto marginal de 15Ω, que es exactamente el tipo de falla que buscamos. Lección 4 del proyecto.

0.3a — Drivers (6 líneas de señal)

PIN ESP32	PIN DEL MÓDULO	ESPERADO	LECTURA
☐ `P27`	IBT-2 **#1** `R_EN` y `L_EN`	<2Ω a **los dos**	
☐ **`P19`**	IBT-2 **#1** `RPWM`	<2Ω	
☐ `P13`	IBT-2 **#1** `LPWM`	<2Ω	
☐ `P4`	IBT-2 **#2** `R_EN` y `L_EN`	<2Ω a **los dos**	
☐ `P16`	IBT-2 **#2** `RPWM`	<2Ω	
☐ `P17`	IBT-2 **#2** `LPWM`	<2Ω	
☐ Riel 5V	`VCC` de **cada** IBT-2	<2Ω	
☐ Soldadura `–`	`B–` de **cada** IBT-2	<2Ω	

`P14` tiene que estar LIBRE. Verificar que no sale ningún cable de ese pin. `RPWM` M1 se movió a `P19` porque `P14` saca señal durante el boot del ESP32 — misma regla que `P5` : no usarlo para nada que mueva un motor.

0.3b — Encoders (4 canales + alimentación)

Medir en el **GPIO del ESP32** y en el **extremo del cable del encoder**. Mapeo de colores confirmado en §5 del registro.

PIN ESP32	CABLE DEL ENCODER	ESPERADO	LECTURA
☐ `P32`	M1 · **amarillo** · canal A	<2Ω	
☐ `P33`	M1 · **verde** · canal B	<2Ω	
☐ `P25`	M2 · **amarillo** · canal A	<2Ω	
☐ `P26`	M2 · **verde** · canal B	<2Ω	
☐ 3V3 (f.3 / f.12)	**azul** de **los dos** encoders	<2Ω	
☐ Riel `–`	**negro** de **los dos** encoders	<2Ω	

⚠️ **Los cables rojo y blanco del encoder NO van a ningún GPIO.** Comparten pestaña con la potencia del motor. Chequeo de seguridad: **ningún GPIO de encoder debe dar continuidad a las pestañas del motor** — si la da, un cable de señal se conectó a la potencia, y al energizar le entrarían 12V a un pin del ESP32.

0.3c — MPU6050 (GY-521)

PIN ESP32	PIN DEL GY-521	ESPERADO	LECTURA
☐ `P21` (f.17-J)	`SDA`	<2Ω	
☐ `P22` (f.14-J)	`SCL`	<2Ω	
☐ 3V3 (f.3 / f.12)	`VCC`	<2Ω	
☐ Riel `–`	`GND`	<2Ω	

⚠️ **El `VCC` del MPU es el que más importa de los cuatro.** Un GY-521 sin alimentación **sigue contestando por I2C** — se alimenta por los diodos de protección de SDA/SCL a unos μA, responde

`WHO_AM_I` correcto y falla recién en `mpu.begin()`. Costó varias horas en §7.13. Esta medición de continuidad, más la de voltaje de la Etapa 2, son las dos formas de no volver a caer.

0.3d — Alimentación del ESP32

DESDE	HASTA	ESPERADO	LECTURA
☐ Riel 5V (Buck `OUT+`)	`VIN` del ESP32 (f.30-B)	<2Ω	
☐ Riel `–`	`GND` del ESP32 — **pin real, fila 25 (A–E)**, entre GPIO12 y GPIO13	<2Ω	
☐ Riel `–`	Pin junto a `VIN` (fila 29) — ⛔ **la serigrafía dice GND y NO es masa**	**abierto** (MΩ)	

Estas dos son las que ya fallaron una vez cada una: el jumper de `VIN` sin contacto real (§3.2, daba 0.7V al energizar) y el `GND` del ESP32 sin unir al riel lógico (§7.6b, motores sin responder mientras el Serial por USB funcionaba perfecto).

⛔ **Y falló una tercera vez (2026-09-21, registro §7.29):** el jumper de masa estaba en un pin que la serigrafía marca GND y **no es masa**. Esta prueba lo dejó pasar porque medía el jumper, no el pin. **Antes de dar por buena la masa, medir el pin elegido contra otro GND de la misma placa (debe dar <1Ω) — la serigrafía no alcanza.**

0.4 — AUSENCIA DE CRUCES

Pares que **tienen que ser nodos distintos**. Es la prueba que caza el reordenamiento M1/M2 mal hecho, y el equivalente del "ENA vs IN1" que ya se usó en §3.2.

⚠️ **Criterio (corregido 2026-09-14, registro §7.27): un cruce da prácticamente 0Ω (<~100Ω). Valores en kΩ son normales** — no se espera "abierto", porque todas las señales llegan a GND por sus pull-downs y las entradas del driver, y encoders/I2C por sus pull-ups. La columna "Normal" es lo medido el 2026-09-14.

PAR	NORMAL	QUÉ DETECTA SI DA ~0Ω
☐ `RPWM` M1 ↔ `LPWM` M1	14.9kΩ	Las dos mitades del puente en corto → el motor nunca gira
☐ `RPWM` M2 ↔ `LPWM` M2	14.9kΩ	Ídem en M2
☐ `RPWM` M1 ↔ `RPWM` M2	14.9kΩ	**Cruce entre lados** — el riesgo nuevo del reordenamiento
☐ `EN` M1 ↔ `EN` M2	11.9kΩ	Los dos drivers habilitados por el mismo GPIO
☐ Canal A ↔ canal B del **mismo** encoder (×2)	6.6kΩ	Cuadratura imposible: no se podría detectar sentido de giro
☐ Canal A de M1 ↔ canal A de M2	~6.6kΩ	Cruce entre encoders — las dos ruedas leerían lo mismo
☐ `SDA` ↔ `SCL`	4.4kΩ	Bus I2C en corto: ningún dispositivo respondería
☐ Cualquier señal ↔ riel 5V	kΩ	Señal pegada a 5V: el driver arrancaría solo
☐ Riel 5V ↔ riel `–`	1.2kΩ	Corto de alimentación
☐ Riel 5V ↔ riel 3V3	24kΩ	5V entrando al riel de 3.3V: mata el MPU y los encoders

Limitación: en circuito esta prueba solo detecta cruces francos, no fugas de decenas de kΩ. Las fugas se verifican energizado, en la Etapa 1.1.

0.5 — RIEL DE 3V3: EL VALOR ESPERADO NO ES "ABIERTO"

MEDICIÓN	ESPERADO	INTERPRETACIÓN
☐ 3V3 (f.3 o f.12) ↔ riel `–`	**≈135kΩ**	Ya medido el 2026-09-12 (§7.25). No es fuga anómala: es el MPU6050 y los encoders compartiendo ese nodo (pull-ups de I2C y diodos de protección). **Si diera abierto**, el MPU o los encoders no están realmente conectados al 3V3.
☐ `EN` del ESP32 (f.13) ↔ riel `–`	abierto	Es la lectura limpia de C_en, que es un cerámico aislado. Un valor bajo = fila mal identificada.

0.6 — MASA DE LOS DRIVERS

MEDICIÓN	ESPERADO	NOTA
☐ `B–` de cada IBT-2 ↔ soldadura `–`	continuidad	Es el **único** camino de masa del driver (V1)
☐ `GND` del header desconectado del riel lógico	verificación **visual**	⚠ **No se puede testear con el multímetro:** `B–` y el `GND` del header son la misma pista dentro del módulo, y `B–` sí está conectado — así que va a dar continuidad igual. Confirmar a ojo que el cable está fuera.
☐ Cables sueltos (`GND` header, `R_IS` , `L_IS`)	aislados **y con brida**	Aislar no alcanza: un conductor suelto en un robot que se va a caer durante el entrenamiento de RL puede tocar `B+`

0.7 — MOTORES Y ENCODERS

MEDICIÓN	ESPERADO	SI FALLA
☐ `M+` ↔ `M−` de cada motor, **desconectado de la bornera**	**finito** (pocos Ω a ~100Ω)	Abierto = cable cortado o terminal floja. Medido: M1 ~9Ω, M2 28–116Ω
☐ Encoder: `VCC` ↔ `GND` de cada uno	no corto	Corto = cable pinchado. Medido: 26kΩ

⚠️ **Criterio corregido 2026-09-16 (registro §7.27).** Esta prueba detecta un **abierto**, no un valor exacto. Con un multímetro, un motor de escobillas quieto no da la resistencia del bobinado: domina el contacto escobilla–conmutador, que cambia con la posición del rotor y baja de a poco con la corriente del instrumento. El 3.5Ω histórico no sirve como criterio.

Cómo medir: motor desconectado, rueda **quieta** (si gira, el motor genera tensión y la lectura no vale), esperar 15–30 s. Falla = abierto en todas las posiciones del rotor.

Etapa 1 · Primera energización · MOTORES DESCONECTADOS

DESCONECTAR `M+` / `M−` DE LAS DOS BORNERAS antes de dar llave

Por qué desconectar los motores. Con los motores fuera, ni una falla total de lógica puede mover el robot. Cuesta cuatro tornillos y elimina por completo el riesgo de esta etapa — que es la primera vez que este cableado ve 12V.

MEDICIÓN	ESPERADO	SI FALLA
☐ Switch V+ en **ON**	—	Tercera causa histórica del síntoma "no se mueve nada"
☐ `B+` IBT-2 **#1** vs GND de potencia	≈12V	Cubre switch, fusible, batería y soldadura de una sola medición. **Son DOS entradas, medir las dos** — no alcanza con una
☐ `B+` IBT-2 **#2**	≈12V	
☐ `VCC` IBT-2 **#1**	≈5V	Sin esto la lógica del driver no conmuta nada aunque el firmware reporte bien — el bug de §7.6, que ya pasó tres veces
☐ `VCC` IBT-2 **#2**	≈5V	
☐ Buck `OUT+` bajo carga	**5.01V**	Si cae mucho, el Buck no sostiene el consumo. Valor de la unidad nueva; el 4.98V era la destruida
☐ `VIN` del ESP32 (f.30-B)	4.8–5.0V	**Una lectura intermedia tipo 0.7V = circuito abierto con fuga**, no conexión parcial. Es exactamente cómo falló el jumper de VIN en §3.2
☐ `3V3` del ESP32 (f.3/f.12)	3.2–3.4V	Si está bajo, revisar el margen de dropout del regulador interno. ✅ **3.35V (2026-09-21).** El 3.515–3.6V anterior era un desfase: el GND del ESP32 estaba en un pin que no es masa (registro §7.29). **Si vuelve a salir alto, verificar primero la masa del ESP32**

1.1 — LA PRUEBA MÁS IMPORTANTE DE TODAS: LAS PULL-DOWNS EN CONDICIÓN REAL

> ⚠️ **Corregido 2026-09-14 (registro §7.27): medir "~0V" NO prueba que la pull-down esté.** Una línea flotante, sin pull-down, medida con un multímetro de ~10MΩ también da ~0V, porque la entrada del multímetro hace de pull-down. Por eso esta prueba tiene dos partes, y hay que pasar las dos. Si una línea falla en cualquiera, **no seguir**: la protección por hardware no está funcionando.

Parte A — ESP32 sin manejar los pines (Test 12 en modo `z`, o sin firmware de motores): detecta fugas hacia 5V/3V3. Una señal que no da ~0V está siendo tirada hacia arriba por algo. ⚠️ Si el ESP32 tiene cargado un sketch viejo de motores, una línea en alto puede ser el firmware y no una falla.

CHEQUEO	ESPERADO	NOTA
☐ Las 6 líneas de señal vs GND	**< 1.1V** (medido 0.07–0.40V)	1.1V = umbral bajo mínimo garantizado del BTS7960 (Infineon 4.4.6). Detecta fugas o cruces hacia arriba. No detecta una pull-down ausente

Parte B — GPIO en `INPUT_PULLUP`: prueba que la pull-down está. El pull-up interno del ESP32 es débil (decenas de kΩ, valor exacto sin confirmar en datasheet) y pierde contra la pull-down de 10kΩ. ✅ **Modo `p` del Test 12** (agregado 2026-09-16, compila, sin flashear): `p` y después el número de línea (1–6). Pone esa línea en `INPUT_PULLUP` con las otras 5 en bajo, e imprime la lectura interna (`digitalRead`) — el multímetro manda.

CHEQUEO	ESPERADO	NOTA
☐ Las 6 líneas con `INPUT_PULLUP`, de a una, vs GND	**claramente bajo**	~3.3V = la pull-down no está o no llega a GND

CHEQUEO	ESPERADO	NOTA
☐ Temperatura tras 1 min: los 2 módulos y el Buck	frío / tibio	Algo caliente con los motores desconectados = corriente que no debería existir. Cortar.

Etapa 2 · ESP32 corriendo · motores todavía desconectados

MOTORES AÚN FUERA · solo sensores y verificación de mapeo

PRUEBA	ESPERADO	POR QUÉ
☐ `esp_reset_reason()` al arrancar	no `ESP_RST_BROWNOUT` (6)	**Nunca se hizo** y está sugerido desde hace tiempo. Si da brownout, el riel se hunde — y explicaría el pendiente del AP que se cae con batería sola
☐ Test 0 — scanner I2C	MPU en `0x68`	Distingue "no hay nadie" de "está en otra dirección"
☐ **`VCC` del MPU con multímetro**	≈3.3V	⚠️ **Que conteste por I2C NO prueba que esté alimentado** — puede vivir de los diodos de protección de SDA/SCL a ~µA. Costó varias horas en §7.13. Medirlo, o mirar el LED del módulo
☐ Test 3 — girar las ruedas **a mano**	4 canales cuentan; M1 y M2 dan **signo opuesto**	Valida los 4 canales sin energizar motores. El signo opuesto es por diseño (montaje en espejo)

2.1 — VERIFICACIÓN DE MAPEO DE SEÑAL, SIN MOVER NADA

La prueba que cierra definitivamente el riesgo del reordenamiento. Poner cada una de las 6 líneas en alto **de a una** y medir en el pin del header del módulo correspondiente. Con los motores desconectados no hay consecuencia posible, y se verifica el mapeo completo — incluido que `P19` llegue al `RPWM` de M1 y no a otro lado.

✅ **Sketch listo:** `test/test12_mapeo_senales_esp32/` (creado 2026-09-13, compila OK). Menú por Serial a 115200: `1` – `6` pone una línea en alto a la vez e imprime dónde poner la punta del multímetro y qué esperar; `0` vuelve al estado seguro. **Nunca hay más de una línea en alto**, y por la tabla de verdad del BTS7960 una sola no puede hacer girar nada (`EN` solo = brake; `RPWM` / `LPWM` solo con `EN` =0 = stand-by) — igual correrlo con los motores desconectados.

⚠️ **El modo `z` NO reemplaza a la Etapa 1.1** (corregido 2026-09-14, registro §7.27). Pone los 6 GPIO en alta impedancia (`INPUT`), pero una línea flotante sin pull-down también mide ~0V con el multímetro. La prueba válida es la parte B de la 1.1 (`INPUT_PULLUP`): modo `p` del mismo sketch. El sketch además imprime `esp_reset_reason()` al arrancar, que es otro ítem de la Etapa 2.

Etapa 3 · Motores conectados · primer movimiento

RUEDAS EN EL AIRE · un motor por vez · duty bajo

⚠️ **El cap de PWM es 121, NO 160.** El 160 se calibró contra los BJT del L298N, que se comían varios volts. El BTS7960 es MOSFET (R_ON 7–10 mΩ): al mismo duty le llega bastante más tensión al motor. Arrancar en 121 y no subirlo sin medir.

PRUEBA	ESPERADO	NOTA
☐ Test 11 — bring-up BTS7960, M1 solo	gira, sin humo, sin calor	Sus `#define` se corrigieron el 2026-09-13 (`M1_RPWM` 14→19)
☐ Test 11 — M2 solo	ídem	
☐ Sentido de giro de cada motor	el esperado	Al recablear a `M+` / `M−` se perdió la corrección de OUT1/OUT2 que se había hecho para M1 — puede hacer falta invertir de nuevo
☐ Signo del encoder vs. sentido comandado	coherente	Cierra el lazo sensor↔actuador

Recién después de esto: medir **coast vs. brake** en el BTS7960 con balanza (hoy es solo inferencia de datasheet, §8.7, y mueve `K_U` por un factor ~2.7), y después el barrido de `K_U` con los **dos motores en brake conduciendo a la vez** — la condición real, no un motor aislado. Con eso sale `θ_umbral` vigente.

Robot Balanceador — creado 2026-09-13. Complementa el checklist de §8.8 del registro y el de "Checklist antes de cualquier prueba que mueva motores" de `CLAUDE.md`, adaptados al cableado rearmado del 2026-09-12. Filas del protoboard: `conexiones-registro-pruebas.md` §3.1b.